If you want to assign an object to a has_many association without saving the object, use the collection.build method.

4.4 has_and_belongs_to_many Association Reference

The has_and_belongs_to_many association creates a many-to-many relationship with another model. In database terms, this associates two classes via an intermediate join table that includes foreign keys referring to each of the classes.

4.4.1 Methods Added by has_and_belongs_to_many

When you declare a has_and_belongs_to_many association, the declaring class automatically gains 16 methods related to the association:

- collection(force_reload = false)
- collection<<(object, ...)
- collection.delete(object, ...)
- collection.destroy(object, ...)
- collection=(objects)
- collection_singular_ids
- collection_singular_ids=(ids)
- collection.clear
- collection.empty?
- collection.size
- collection.find(...)
- collection.where(...)
- collection.exists?(...)
- collection.build(attributes = {})
- collection.create(attributes = {})
- collection.create!(attributes = {})

In all of these methods, collection is replaced with the symbol passed as the first argument to has_and_belongs_to_many, and collection_singular is replaced with the singularized version of that symbol. For example, given the declaration:

```
class Part < ActiveRecord::Base
  has_and_belongs_to_many :assemblies
end
```

Each instance of the Part model will have these methods:

```
assemblies(force_reload = false)
assemblies<<(object, ...)
assemblies.delete(object, ...)
assemblies.destroy(object, ...)
assemblies=(objects)
assembly_ids
assembly_ids=(ids)
assemblies.clear
assemblies.empty?
assemblies.size
assemblies.find(...)
assemblies.where(...)
assemblies.exists?(...)
assemblies.build(attributes = {}, ...)
assemblies.create(attributes = {})
assemblies.create!(attributes = {})
```

4.4.1.1 Additional Column Methods